A Comparative Performance Analysis of a Two Dimensional Diffusion Model Using Different Compiler Optimisation Techniques

Exam No. B024703, Group 03

November 24, 2016

1 Introduction

This report details experiments undertaken to compare the performance achieved using two different compilers, The GNU C Compiler and the PGI C Compiler, on a model using different terrain sizes. The compilers were used with different optimisation flags and results were gathered using various sizes of terrain.

2 Methods

2.1 The Source Code

The source consists of code written in C99. It also includes a suite of unit tests written using the CUnit framework. These tests were used to verify the stability of each compilation technique.

The code, called CLiPH or Circle of Life for Pumas and Hares, computes a diffusion model of predators and prey. It takes as an input a map which contains $M \cdot N$ elements, where M is height and N is width. Each of these elements is either land or sea. No predators or prey can exist in areas designated as sea. As an output it calculates an average population density for

both the predators and the prey as well as generating separate population density maps as PPM files for both classes of animal.

Among other things, the frequency with which these outputs are generated are defined by parameters stored in a configuration file. As the PPM output files can be large where N and M are also large, a value was selected in this configuration file so that these maps would never be output. This allows the performance analysis here to focus on compute time, rather than the I/O that would be involved in writing these large files to disk. However, the calculation of a mean population density involved some not inconsiderable computation. These densities where therefore calculated every 100 timesteps. The parameters were kept constant throughout all experiments.

2.2 Compilation

The source code was compiled using two different compilers. These were the GNU C Compiler and the PGI C compiler. The source was compiled on and for an Intel quad core Core-i5 CPU. Full CPU details and compiler versions are included as an appendix.

In both cases the source was compiled first with no optimisation flags, then with the `-O1` `-O2` and `-O3` flags. These flags specify differing degrees of compiler optimisation.

2.3 Input Maps

To create a map to use as input for the program, an image in black and white was created with the GNU Image Manipulation Program (GIMP). In this black represented land and white sea. For the sake of simplicity the image was a square where $M = N = 1000$. This image image was then resized using the ImageMagick tool to sizes between 10% and 400% of the original, with increments of 10%. Finally the images were converted to a ASCII map format readable by the CLiPH program using a small utility that converts darker shades to 1 and lighter shades to 0. This resizing of the same image was done to ensure that the ratio of land to sea would remain constant between simulations.

2.4 Running Timed Simulations

The variously sized images were used as inputs for the different executables generated in the compilation part of the experiment. These were timed using a timing feature built into the CLiPH program.

For the sake of repeatability all stages of the experiment were conducted programmatically using a Bash script and the Make utility for compilation as well as gnuplot for displaying results. These scripts are available on request from the author and would make conducting the same experiments on different processors trivial.

3 Results

3.1 Compilation

Both GNU and The Portland Group, the authors of the two compilers, warn that using high levels of compiler optimisation increase compilation times and memory usage [1][2]. However, as in all cases compilation times were imperceptible to a human user, it was considered unnecessary to accurately measure or discuss further compilation time or memory use.

3.2 Stability and Correctness

In all cases the code was compiled with no warnings or errors from the compiler and the suite of unit tests passed completely. Beyond this it is hard to compare with surety the accuracy of the results generated, as the initial population densities are generated pseudo-randomly. This means that accuracy cannot be verified by repetition. It was noted that credible results were generated in all cases, with populations of predator and prey oscillating in a slightly out of phase manner, as would be expected.

The fully passing unit tests along with somewhat unscientific inspection of the output population densities convinced the author that correctness should not be a factor when comparing compilers, or the compiler optimisation techniques, used on this code.

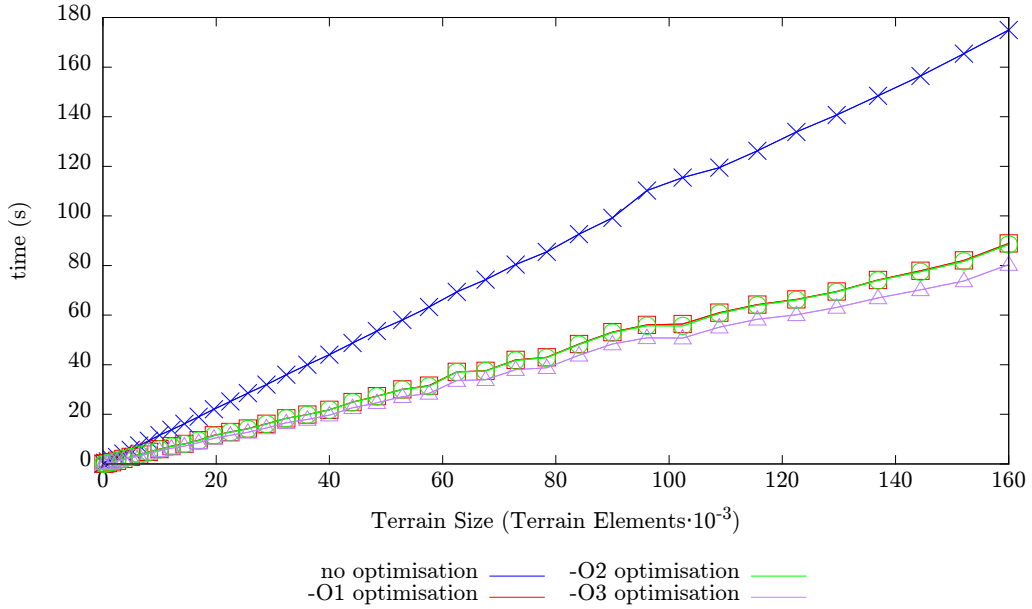

Figure 1: Graph showing GCC execution times. The executable with no optimisation takes significantly longer than the others.

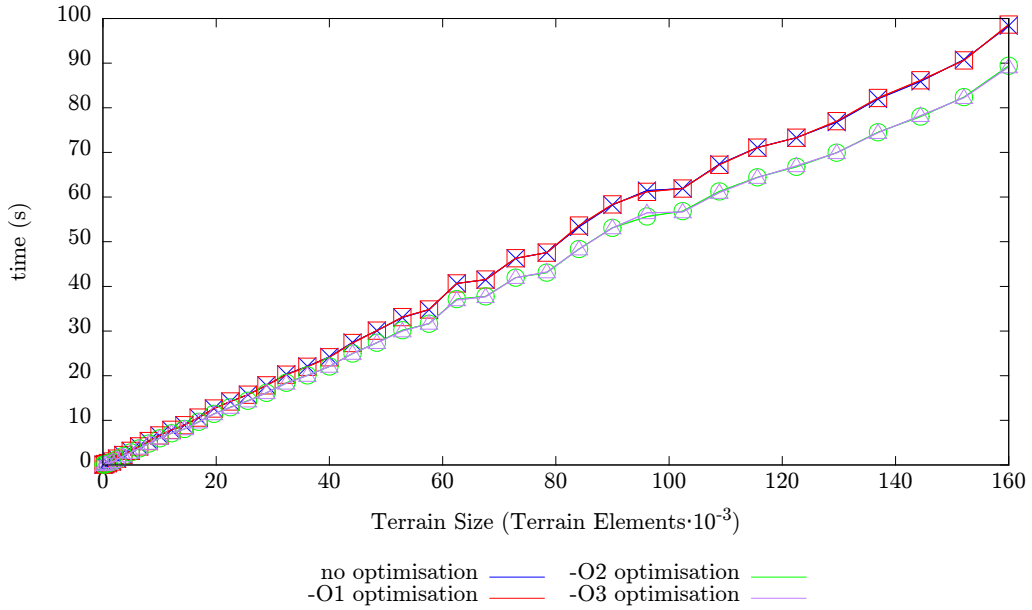

Figure 2: Graph showing PGI execution times on different sized terrains. Note that the results are split into two.

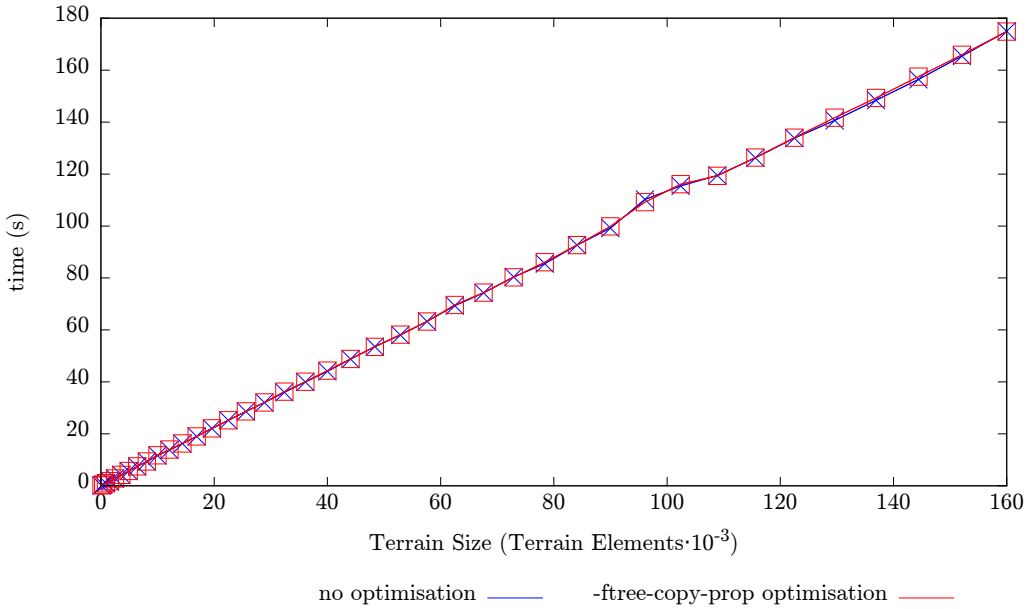

Figure 3: Graph showing performance with the GNU Compiler with and without the -ftree-copy-prop flag. Note performance is identical in both cases.

3.3 Performance Characteristics

In broad terms, for both compilers and all optimisation flags, the time taken to complete a simulation increased linearly with the number of terrain elements ($M \cdot N$). This is what would be expected, as the size of the terrain is proportional to the number of calculations required to compute the simulation. It could therefore be said that the CLiPH program is an $O(n)$ algorithm.

There are, however some irregularities in this linear relationship between compute time and terrain size. In all cases but the unoptimised GCC executable, there is a small bump in compute time as the number of elements reaches 60,000. This could be explained by a caching effect. The CPU used had 6 MiB of L3 cache. As each element on the terrain was represented by a 64 bit double precision floating point number, a terrain modelled as a 2 dimensional array of doubles would consume this 6 MiB of cache. However, it is then surprising that no non-linear performance decrease is observable at a figure half of this, as the simulation depends on 2 arrays of this size.

For the GNU compiler, the results, shown in figure 1, show that with no op-

timisation execution times are longer than those of the various optimisation degrees by approximately a factor of two. In the source code, in each iteration of the simulation, a new 2 dimensional array is calculated from the last, using a finite difference scheme. After this is done the data in the new array is explicitly copied element by element from the old to the new. This process is, however, unnecessary, as the pointers to these two arrays could simply be swapped. Although no calculations are performed on this copying pass, if memory access is the completely dominant factor in this computation, doing this unnecessary copy could almost double the execution time. As described in [1] all optimisation options up from `-O1` include the `-ftree-copy-prop` option. This option removes unnecessary memory copies. If this hypothesis is correct, then it would not only show that this specific optimisation property was responsible for the drastic performance improvement, but also that memory access was the dominant factor for performance with this code.

To verify this hypothesis the experiment was run again using just the option to remove unnecessary copies. This result, show in figure 3, shows no difference in performance between the unoptimised code and the code optimised using the memory copy reduction flag. This hypothesis is therefore shown to be incorrect.

For both compilers, apart from the unoptimised GCC case, we see there are two distinct groupings of results, with one approximately 10% faster than the other. This separation occurs between the `-O2` and `-O3` optimisation levels with the GNU compiler and between the `-O1` and `-O2` levels on the PGI compiler. It seems likely that there is an optimisation that increases the performance by this specific amount. As the `-O3` flag switches on five optimisation techniques, it goes beyond the scope of this report to try each and find which is responsible for this performance increase. The Portland Group do not release details of specific techniques used by each optimisation level.

4 Conclusion

In conclusion, it has been shown that using aggressive compiler optimisation was, in this case, a very worthwhile endeavour. In extreme cases the performance increased by approximately a factor of two with no detectable degradation of correctness. For this code the compile times and executable file sizes were too small to be considered as important factors on modern

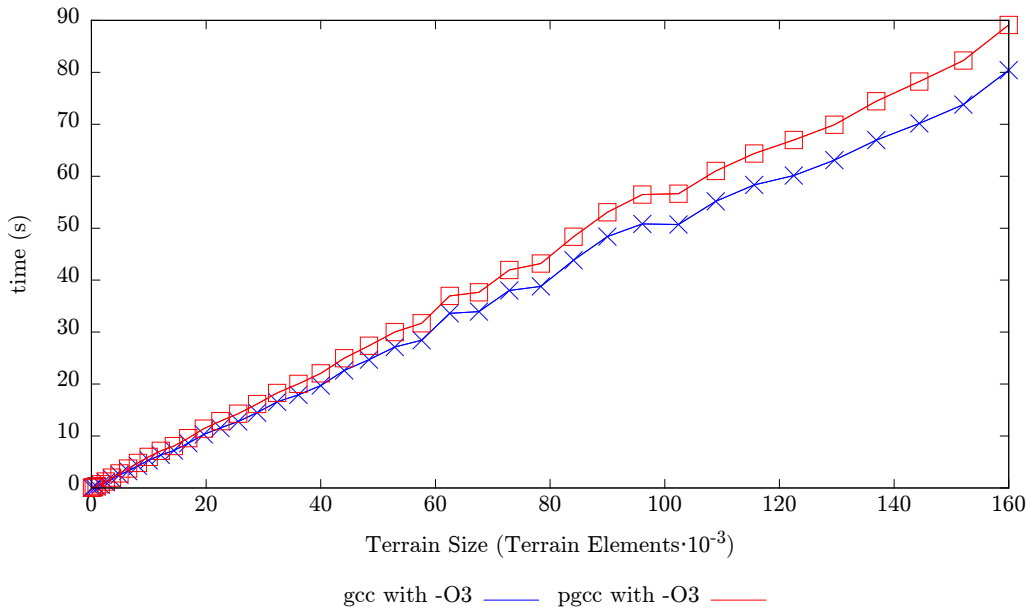

Figure 4: Graph showing performance in the most highly optimised case for both compilers. The GNU compiled code is faster, but both have the same shape curve. Note particularly the bump around $60 \cdot 10^3$ which could be explained by L3 caching effects.

hardware.

It remains unclear in the case of The GNU C Compiler what causes the drastic increase in performance after the `-O1` flag is used. It was shown that this performance improvement was not caused by the `-ftree-copy-opt` optimisation option. Further experiments could be undertaken to find what optimisation technique was responsible. Further work could also be undertaken to identify the root causes, or cause, of the improvement gained between the `-O2` and `-O3` flags for GCC and `-O1` and `O2` for GPCC. However, this goes beyond the scope of this report.

Caching and other memory hierarchy effects were minor and unconfirmed, although non-linearities in execution times against terrain size were largely consistent between experiments and warrants further investigation.

As shown clearly in figure 4, for the maximally optimised cases the GNU C Compiler gives slightly better performance than the PGI C Compiler.

References

[1] GNU. *Optimize Options* `https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html` Accessed: 14-11-2016

[2] The Portland Group *PGI Compiler User's Guide for Intel 64 and AMD64C PUs* `http://www.pgroup.com/doc/pgiug-x64.pdf` Accessed: 14-11-2016

A Details of Processor and Compilers

A.1 Processor

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                4
On-line CPU(s) list:   0-3
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):             1
NUMA node(s):          1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 60
Model name:            Intel(R) Core(TM) i5-4570S CPU @ 2.90GHz
Stepping:              3
CPU MHz:               1200.667
BogoMIPS:              5786.47
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              6144K
NUMA node0 CPU(s):     0-3
```

A.2 PGI C Compiler

```
pgcc 15.5-0 64-bit target on x86-64 Linux -tp haswell
The Portland Group - PGI Compilers and Tools
```

A.3 GNU C Compiler

```
gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-4)
```